

PL/SQL

1. Introduction to PL/SQL

What is PL/SQL?

PL/SQL (Procedural Language/Structured Query Language) is Oracle's procedural extension to SQL.

PL/SQL allows you to write code that defines *how* to process that data using traditional programming logic.

Why PL/SQL?

SQL alone cannot perform complex business logic efficiently.

PL/SQL helps to:

- Reduce network traffic
- Improve performance
- Handle errors
- Create reusable code
- Build enterprise applications

SQL	PL/SQL
Declarative language	Procedural language
Executes one statement at a time	Executes block of statements
Cannot use loops	Supports loops
Cannot handle exceptions	Supports exception handling
No variables	Supports variables
Used for data operations	Used for business logic

SQL

```
SELECT salary FROM employee WHERE emp_id=101;
```

PL/SQL

```
DECLARE
    sal NUMBER;
BEGIN
    SELECT salary INTO sal
    FROM employee
    WHERE emp_id=101;

    DBMS_OUTPUT.PUT_LINE(sal);
END;
/
```

2. PL/SQL Environment

PL/SQL Block Structure

Every PL/SQL program is written inside a block.

Structure

```
DECLARE
    -- Variable declarations
BEGIN
    -- Executable statements
EXCEPTION
    -- Error handling
END;
/
```

Components

DECLARE

Optional section.

Used for:

- Variables
- Constants
- Cursors

```
DECLARE
    name VARCHAR2(20);
```

BEGIN

Mandatory section.

Contains executable statements.

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello');
END;
/
```

EXCEPTION

Optional section.

Handles runtime errors.

```
EXCEPTION
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error');
```

Anonymous Block

No name.

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello');
END;
/
```

Used for:

- Testing

- Temporary programs
-

Named Block

Stored in database.

Examples:

- Procedure
- Function
- Trigger
- Package

```
CREATE OR REPLACE PROCEDURE greet
IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello');
END;
/
```

3. Basic PL/SQL Syntax

Variables

```
DECLARE
age NUMBER;
BEGIN
age := 21;
END;
/
```

Data Types

NUMBER

```
salary NUMBER(10,2);
```

Example:

```
salary NUMBER :=25000;
```

VARCHAR2

Stores strings.

```
name VARCHAR2(30);
```

Example:

```
name := 'Sanjay';
```

CHAR

Fixed-length character.

```
genderCHAR(1);
```

DATE

Stores date and time.

```
joining_dateDATE;
```

BOOLEAN

```
flagBOOLEAN :=TRUE;
```

Assigning Values

```
age :=22;
```

Example:

```
DECLARE
    name VARCHAR2(20);
BEGIN
    name := 'Sanjay';
    DBMS_OUTPUT.PUT_LINE(name);
END;
/
```

4. Controlled Structure

Simple IF

```
IF age > 18 THEN
    DBMS_OUTPUT.PUT_LINE('Adult');
END IF;
```

IF ELSE

```
IF marks >= 50 THEN
    DBMS_OUTPUT.PUT_LINE('Pass');
ELSE
    DBMS_OUTPUT.PUT_LINE('Fail');
ENDIF;
```

IF ELSIF

```
IF marks >= 90 THEN
    grade := 'A';
ELSIF marks >= 75 THEN
    grade := 'B';
ELSE
```

```
grade := 'C';  
ENDIF;
```

CASE Statement

```
CASE grade  
WHEN 'A' THEN  
    DBMS_OUTPUT.PUT_LINE('Excellent');  
WHEN 'B' THEN  
    DBMS_OUTPUT.PUT_LINE('Good');  
ELSE  
    DBMS_OUTPUT.PUT_LINE('Average');  
ENDCASE;
```

Loops

Simple LOOP

```
DECLARE  
    i NUMBER:=1;  
BEGIN  
    LOOP  
        DBMS_OUTPUT.PUT_LINE(i);  
        i:=i+1;  
  
    EXITWHEN i>5;  
    ENDLOOP;  
END;  
/
```

Output:

```
1  
2  
3  
4  
5
```

WHILE Loop

```
DECLARE
    i NUMBER:=1;
BEGIN
    WHILE i<=5LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        i:=i+1;
    ENDLOOP;
END;
/
```

FOR Loop

```
BEGIN
    FOR i IN 1..5LOOP
        DBMS_OUTPUT.PUT_LINE(i);
    ENDLOOP;
END;
/
```

Output:

```
1
2
3
4
5
```